Day 07

Collections.sort() → method present in the collection class

ex-> 10, 1, 2, 35, 58

1, 2, 10, 35, 58

ex -> Jenny, Pradeep, Ankur

Ankur, Pradeep, Jenny

Java List Sorting

Collections.sort() • List.sort() • Comparable • Comparator

Learning goal: Understand how Java sorts lists, why the element type matters, and when to use natural ordering versus a custom comparator.

1. Sorting with Collections.sort()

Collections is a utility class in `java.util`. `sort()` is static, so call it with the class name. The no-comparator overload sorts elements using their **natural ordering**.

```
import java.util.*;

List<Integer> l1 = new ArrayList<>(List.of(2, 0, 23, 14, 7));
Collections.sort(l1);
System.out.println(l1);
```

Output: [0, 2, 7, 14, 23]

Conceptually, the method has this signature:

```
public static <T extends Comparable<? super T>>
void sort(List<T> list)
```

The type bound means the elements must support comparison through `Comparable` (or a suitable supertype). `Integer` implements `Comparable<Integer>`, so Java knows how to order integers.

2. Sorting with List.sort()

`List.sort()` is an instance method. It requires one argument: a `Comparator`. Its signature is:

```
void sort(Comparator<? super E> c);
```

Calling `l3.sort()` without an argument is a compilation error. Pass `null` to request natural ordering, or provide a comparator.

```
l3.sort(null);                          // Natural ordering
l3.sort(Comparator.naturalOrder());     // Natural ordering
l1.sort(Comparator.reverseOrder());     // Descending integer order
```

Note: The list must support the required modifications for sorting. For example, `List.of(...)` creates an unmodifiable list; copy it into an `ArrayList` before sorting.

3. Complete String example

```
List<String> l2 = List.of("bkk", "aab", "aac", "baa");
List<String> l3 = new ArrayList<>(l2);

// Static utility method
Collections.sort(l3);
System.out.println(l3);

// Instance method; null means natural ordering
l3.sort(null);
System.out.println(l3);
```

Output:

```
[aab, aac, baa, bkk]
[aab, aac, baa, bkk]
```

Strings implement `Comparable<String>`. Their natural order is lexicographic: Java compares characters from left to right until a difference is found.

4. Comparable vs Comparator

Feature	Comparable	Comparator
Purpose	Defines a class's natural ordering.	Defines an alternative/custom ordering.
Package	java.lang	java.util
Main method	compareTo(other)	compare(a, b)
Implemented by	Usually the element class itself.	A separate comparator class, lambda, or built-in comparator.
Used for	Default/natural sorting.	Custom sorting, e.g. descending order or sorting by a field.

```
Integer a = 10;
Integer b = 20;
System.out.println(a.compareTo(b)); // Negative value
```

`compareTo()` returns a negative value when the first value is less than the second, zero when they are equal, and a positive value when the first is greater.

5. Does List implement Comparable?

No. The list itself does not need to implement `Comparable` for its elements to be sorted. `Integer` and `String` implement `Comparable`; those element types provide natural ordering. The no-comparator sorting methods rely on that ordering.

If you define a custom class (for example, Student), implement `Comparable<Student>` and define `compareTo()` to establish its natural ordering, or use a `Comparator<Student>` when you want a separate/custom rule.

6. Quick revision table

Method	Call	Ordering
Collections.sort(list)	Static method on Collections	Natural order
Collections.sort(list, c)	Static method with comparator	Custom order
list.sort(null)	Instance method on List	Natural order
list.sort(c)	Instance method with comparator	Custom order

Remember: `Collections.sort(list)` is a static utility call; `list.sort(comparator)` is an instance call. Natural ordering comes from the elements' `Comparable` implementation, while custom ordering comes from a `Comparator`.

To make custom objects

```
class Student implements Comparable<Student>{  7 usages
    private String name;  1 usage
    private double marks;  1 usage
    private int rollNumber;  3 usages
    Student(String name,double marks,int rollNumber){  3 usages
        this.name=name;
        this.marks=marks;
        this.rollNumber=rollNumber;
    }

    @Override
    public int compareTo(Student that) {// that is object only
        return this.rollNumber- that.rollNumber;// this will do comparison w.r.t rollNu
    }
}
// now class is make now make the custom objects in the main class
```

this method of compare is not good

return Integer.compare(this.rollNumber, that.rollNumber);

```
53          // we gona to make a custom objects
54
55          List<Student>studentList=List.of(new Student( name: "bkk", marks: 98.45, rollNumber: 32),
56                  new Student( name: "ram", marks: 97.45, rollNumber: 111),
57                  new Student( name: "sham", marks: 87.23, rollNumber: 112)
58                  );
59      // now it is immutable list to make it mutable
60          List<Student>sl=new ArrayList<>(studentList);
61      //      Collections.sort(s1);
62      //      java: cannot find symbol
63      //      symbol:   variable s1
64      //      location: class DAy07
65          // mow this will give error as java does not know how to compare the student object
66          // which is not the case of the wrapper classes
67          // so class has to implement the comparable interface
68
69          Collections.sort(sl);
70          System.out.println(sl);
71
72      }
73  }
74
```

```
[Student@5b480cf9, Student@6f496d9f, Student@723279cf]

Process finished with exit code 0
```

to correct this we have to override toString method do it

```
    @Override
    public String toString(){
        return rollNumber+" "+name+" "+marks;
    }
}
```

```
Collections.sort(sl);
System.out.println(sl);
//[32 bkk 98.45, 111 ram 97.45, 112 sham 87.23]
```